

Notas para el curso de Métodos Numéricos

Carlos E Martínez-Rodríguez

23 septiembre, 2025

Índice general

1. Métodos de eliminación	1
1.1. Gaussiana Simple	1
1.2. Gaussiana con pivoteo parcial	2
1.3. Gauss Jordan	3
1.4. Cálculo de Inversa	5
1.5. Factorización LU	6
1.6. Factorización de Cholesky	9
1.7. Solución vía Cholesky	10
2. Métodos Iterativos	11
2.1. Gauss Jacobi	11
2.2. Gauss Seidel	12

Capítulo 1

Métodos de eliminación

1.1. Gaussiana Simple

1.1.1. Ejemplo

```
## k=1, i=2, m=-1.5
##          b
## [1,]  2 1.0 -1.0  8
## [2,]  0 0.5  0.5  1
## [3,] -2 1.0  2.0 -3
## k=1, i=3, m=-1
##          b
## [1,]  2 1.0 -1.0  8
## [2,]  0 0.5  0.5  1
## [3,]  0 2.0  1.0  5
## k=2, i=3, m=4
##          b
## [1,]  2 1.0 -1.0  8
## [2,]  0 0.5  0.5  1
## [3,]  0 0.0 -1.0  1

## [1]  2  3 -1

##
## [1,]  2 1.0 -1.0
## [2,]  0 0.5  0.5
## [3,]  0 0.0 -1.0

##          b
## [1,]  2 1.0 -1.0  8
## [2,]  0 0.5  0.5  1
## [3,]  0 0.0 -1.0  1

##      [,1]
## [1,]    8
## [2,]   -11
## [3,]    -3
```

1.2. Gaussiana con pivoteo parcial

```
gauss_piv_parcial <- function(A, b, tolerancia, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); if (ncol(A) != n) stop("A debe ser cuadrada.")
  if (length(b) != n) stop("Dimensiones de b incorrectas.")
  Ab <- cbind(A, b)
  for (k in 1:(n-1)) {
    # Selección del pivote (máximo en valor absoluto en la columna k desde la fila k)
    max_row <- which.max(abs(Ab[k:n, k])) + (k - 1)
    if (abs(Ab[max_row, k]) < tolerancia){stop(sprintf("Pivote casi nulo en columna%d", k))}
    # Intercambio de filas si es necesario
    if (max_row != k) {
      Ab[c(k, max_row), ] <- Ab[c(max_row, k), ]
      if (verbose) {cat(sprintf("Intercambio de fila%d con fila%d\n", k, max_row)); print(Ab)}
    }
    for (i in (k + 1):n) {
      m <- Ab[i, k] / Ab[k, k]; Ab[i, k:ncol(Ab)] <- Ab[i, k:ncol(Ab)] - m * Ab[k, k:ncol(Ab)]
      if (abs(Ab[i, k]) < tolerancia) Ab[i, k] <- 0
      if (verbose) {cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m)); print(Ab)}
    }
  }
  x <- numeric(n)
  for (i in n:1) {
    if (i == n) {
      suma <- 0
    } else {suma <- sum(Ab[i, (i + 1):n] * x[(i + 1):n])}
    x[i] <- (Ab[i, n + 1] - suma) / Ab[i, i]
  }
  list(x = x, U = Ab[, 1:n], Ab = Ab)
}
```

1.2.1. Ejemplo

```
## Intercambio de fila 1 con fila 2
##
##      b
## [1,]  1 -2 -3 -3
## [2,]  0  2  1  3
## [3,] -1  1  2 -1
## k=1, i=2, m=0
##
##      b
## [1,]  1 -2 -3 -3
## [2,]  0  2  1  3
## [3,] -1  1  2 -1
## k=1, i=3, m=-1
##
##      b
## [1,]  1 -2 -3 -3
## [2,]  0  2  1  3
## [3,]  0 -1 -1 -4
## k=2, i=3, m=-0.5
##
##      b
```

```
## [1,] 1 -2 -3.0 -3.0
## [2,] 0 2 1.0 3.0
## [3,] 0 0 -0.5 -2.5

## [1] 10 -1 5
```

1.3. Gauss Jordan

```
gauss_jordan <- function(A, b, tolerancia, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A)
  if (!is.null(b) && length(b) != n) stop("Dimensiones incompatibles entre A y b.")
  Ab <- cbind(A, as.matrix(b)); ncols <- ncol(Ab); row <- 1
  for (col in 1:m) {
    if (row > n) break
    pivot_row_rel <- which.max(abs(Ab[row:n, col]))
    pivot_row <- row + pivot_row_rel - 1
    if (abs(Ab[pivot_row, col]) < tolerancia) {
      if (verbose) cat(sprintf("Sin pivote usable en col=%d (|piv|<tol). Se omite columna.\n", col))
      next
    }
    if (pivot_row != row) {
      Ab[c(row, pivot_row), ] <- Ab[c(pivot_row, row), ]
      if (verbose) {cat(sprintf("Swap filas %d <-> %d (col=%d)\n", row, pivot_row, col));print(Ab)}
    }
    pivote <- Ab[row, col]; Ab[row, ] <- Ab[row, ] / pivote
    if (verbose) {cat(sprintf("Normaliza fila %d por pivote %.6g (col=%d)\n", row, pivote, col));print(Ab)}
    for (r in 1:n) {
      if (r == row) next
      factor <- Ab[r, col]
      if (abs(factor) > tolerancia) {
        Ab[r, ] <- Ab[r, ] - factor * Ab[row, ]
        if (verbose) {cat(sprintf("R %d := R %d - (%.6g)*R %d (col=%d)\n", r, r, factor, row, col));print(Ab)}
      }
    }
    row <- row + 1
  }
  Ab[abs(Ab) < tolerancia] <- 0; out <- list(RREF = Ab); X <- Ab[, (m + 1):ncols, drop = FALSE]
  out$x <- if (ncol(X) == 1) as.vector(X) else X
  return(out)
}
```

1.3.1. Ejemplo

```
## Swap filas 1 <-> 2 (col=1)
##      [,1] [,2] [,3] [,4]
## [1,]  -3  -1   2  -11
## [2,]   2   1  -1   8
## [3,]  -2   1   2  -3
## Normaliza fila 1 por pivote -3 (col=1)
##      [,1]      [,2]      [,3]      [,4]
```

```

## [1,] 1 0.3333333 -0.6666667 3.666667
## [2,] 2 1.0000000 -1.0000000 8.000000
## [3,] -2 1.0000000 2.0000000 -3.000000
## R2 := R2 - (2)*R1 (col=1)
## [1,] [2] [3] [4]
## [1,] 1 0.3333333 -0.6666667 3.666667
## [2,] 0 0.3333333 0.3333333 0.6666667
## [3,] -2 1.0000000 2.0000000 -3.0000000
## R3 := R3 - (-2)*R1 (col=1)
## [1,] [2] [3] [4]
## [1,] 1 0.3333333 -0.6666667 3.666667
## [2,] 0 0.3333333 0.3333333 0.6666667
## [3,] 0 1.6666667 0.6666667 4.3333333
## Swap filas 2 <-> 3 (col=2)
## [1,] [2] [3] [4]
## [1,] 1 0.3333333 -0.6666667 3.666667
## [2,] 0 1.6666667 0.6666667 4.3333333
## [3,] 0 0.3333333 0.3333333 0.6666667
## Normaliza fila 2 por pivote 1.66667 (col=2)
## [1,] [2] [3] [4]
## [1,] 1 0.3333333 -0.6666667 3.666667
## [2,] 0 1.0000000 0.4000000 2.6000000
## [3,] 0 0.3333333 0.3333333 0.6666667
## R1 := R1 - (0.3333333)*R2 (col=2)
## [1,] [2] [3] [4]
## [1,] 1 0.0000000 -0.8000000 2.8000000
## [2,] 0 1.0000000 0.4000000 2.6000000
## [3,] 0 0.3333333 0.3333333 0.6666667
## R3 := R3 - (0.3333333)*R2 (col=2)
## [1,] [2] [3] [4]
## [1,] 1 0 -0.8 2.8
## [2,] 0 1 0.4 2.6
## [3,] 0 0 0.2 -0.2
## Normaliza fila 3 por pivote 0.2 (col=3)
## [1,] [2] [3] [4]
## [1,] 1 0 -0.8 2.8
## [2,] 0 1 0.4 2.6
## [3,] 0 0 1.0 -1.0
## R1 := R1 - (-0.8)*R3 (col=3)
## [1,] [2] [3] [4]
## [1,] 1 0 0.0 2.0
## [2,] 0 1 0.4 2.6
## [3,] 0 0 1.0 -1.0
## R2 := R2 - (0.4)*R3 (col=3)
## [1,] [2] [3] [4]
## [1,] 1 0 0 2
## [2,] 0 1 0 3
## [3,] 0 0 1 -1

## [1] 2 3 -1

```


1.4. Cálculo de Inversa

```

gauss_jordan_inversa <- function(A,tolerancia, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A)
  if (n != m) stop("Para invertir, A debe ser cuadrada.")
  Ab <- cbind(A, diag(n)); ncols <- ncol(Ab); row <- 1
  for (col in 1:m) {
    if (row > n) break
    pivot_row_rel <- which.max(abs(Ab[row:n, col]))
    pivot_row <- row + pivot_row_rel - 1
    if (abs(Ab[pivot_row, col]) < tolerancia) {
      stop(sprintf("No hay pivote en la columna%d (|piv|<tol).", col))
    }
    if (pivot_row != row) {
      Ab[c(row, pivot_row), ] <- Ab[c(pivot_row, row), ]
      if (verbose) {cat(sprintf("Intercambia filas%d <-> %d (col=%d)\n", row, pivot_row, col)); print(Ab)}
    }
    pivote <- Ab[row, col]; Ab[row, ] <- Ab[row, ] / pivote
    if (verbose) {cat(sprintf("Normaliza fila%d por pivote%.6g (col=%d)\n", row, pivote, col)); print(Ab)}
    for (r in 1:n) {
      if (r == row) next
      factor <- Ab[r, col]
      if (abs(factor) > tolerancia) {
        Ab[r, ] <- Ab[r, ] - factor * Ab[row, ]
        if (verbose){cat(sprintf("R%d := R%d - (%.6g)*R%d (col=%d)\n", r, r, factor, row, col)); print(Ab)}
      }
    }
    row <- row + 1
  }
  Ab[abs(Ab) < tolerancia] <- 0; LHS <- Ab[, 1:m, drop = FALSE]
  if (!all(LHS == diag(n))) {stop("La matriz es singular o la tolerancia es muy estricta.")}
  Ainv <- Ab[, (m + 1):ncols, drop = FALSE]
  list(Ainv = Ainv, RREF = Ab)
}

```

1.4.1. Ejemplo

```

## Intercambia filas 1 <-> 3 (col=1)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  5    6    0    0    0    1
## [2,]  0    1    4    0    1    0
## [3,]  1    2    3    1    0    0
## Normaliza fila 1 por pivote 5 (col=1)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  1  1.2    0    0    0  0.2
## [2,]  0  1.0    4    0    1  0.0
## [3,]  1  2.0    3    1    0  0.0
## R3 := R3 - (1)*R1 (col=1)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  1  1.2    0    0    0  0.2

```

```
## [2,]    0 1.0    4    0    1 0.0
## [3,]    0 0.8    3    1    0 -0.2
## Normaliza fila 2 por pivote 1 (col=2)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1 1.2    0    0    0 0.2
## [2,]    0 1.0    4    0    1 0.0
## [3,]    0 0.8    3    1    0 -0.2
## R1 := R1 - (1.2)*R2 (col=2)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1 0.0 -4.8    0 -1.2 0.2
## [2,]    0 1.0  4.0    0  1.0 0.0
## [3,]    0 0.8  3.0    1  0.0 -0.2
## R3 := R3 - (0.8)*R2 (col=2)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1    0 -4.8    0 -1.2 0.2
## [2,]    0    1  4.0    0  1.0 0.0
## [3,]    0    0 -0.2    1 -0.8 -0.2
## Normaliza fila 3 por pivote -0.2 (col=3)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1    0 -4.8    0 -1.2 0.2
## [2,]    0    1  4.0    0  1.0 0.0
## [3,]    0    0  1.0   -5  4.0 1.0
## R1 := R1 - (-4.8)*R3 (col=3)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1    0    0  -24  18    5
## [2,]    0    1    4    0    1    0
## [3,]    0    0    1   -5    4    1
## R2 := R2 - (4)*R3 (col=3)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    1    0    0  -24  18    5
## [2,]    0    1    0  20  -15   -4
## [3,]    0    0    1   -5    4    1

##      [,1] [,2] [,3]
## [1,]  -24  18    5
## [2,]   20 -15   -4
## [3,]   -5   4    1

##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1
```

1.5. Factorización LU

1.5.1. Sin Pivoteo

```
lu_simple <- function(A, tol = 1e-12, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A)
  if (n != m) stop("A debe ser cuadrada.")
  U <- A; L <- diag(n)
```

```

for (k in 1:(n - 1)) {
  piv <- U[k, k]
  if (abs(piv) < tol) {stop(sprintf("Pivote casi cero en k=%d. No se puede continuar.", k))}
  for (i in (k + 1):n) {
    m <- U[i, k] / piv; L[i, k] <- m; U[i, k:n] <- U[i, k:n] - m * U[k, k:n]
    if (verbose) {cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m)); print(U)}
  }
}
list(L = L, U = U)
}

```

1.5.1.1. Ejemplo

```

##      [,1] [,2] [,3]
## [1,]  1.0  0   0
## [2,] -1.5  1   0
## [3,] -1.0  4   1

##      [,1] [,2] [,3]
## [1,]    2  1.0 -1.0
## [2,]    0  0.5  0.5
## [3,]    0  0.0 -1.0

```

1.5.2. Con Pivoteo

```

lu_piv_parcial <- function(A, tol = 1e-12, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A); m <- ncol(A); if (n != m) stop("A debe ser cuadrada.")
  U <- A; L <- diag(n); P <- diag(n)
  for (k in 1:(n - 1)) {
    max_row <- which.max(abs(U[k:n, k])) + (k - 1)
    if (abs(U[max_row, k]) < tol) {stop(sprintf("Matriz singular (pivote ~ 0).", k))}
    if (max_row != k) {
      U[c(k, max_row), ] <- U[c(max_row, k), ]; P[c(k, max_row), ] <- P[c(max_row, k), ]
      if (k > 1) {L[c(k, max_row), 1:(k - 1)] <- L[c(max_row, k), 1:(k - 1)]}
      if (verbose) {cat(sprintf("Intercambia filas %d <-> %d\n", k, max_row)); print(U)}
    }
    for (i in (k + 1):n) {
      m <- U[i, k] / U[k, k]; L[i, k] <- m; U[i, k:n] <- U[i, k:n] - m * U[k, k:n];
      if (verbose) {cat(sprintf("k=%d, i=%d, m=%.6g\n", k, i, m)); print(U)}
    }
  }
  list(P = P, L = L, U = U)
}

```

1.5.2.1. Ejemplo

```

##      [,1] [,2] [,3]
## [1,]    0    1    0
## [2,]    1    0    0
## [3,]    0    0    1

```

```
##      [,1] [,2] [,3]
## [1,]    1  0.0    0
## [2,]    0  1.0    0
## [3,]   -1 -0.5    1

##      [,1] [,2] [,3]
## [1,]    1   -2 -3.0
## [2,]    0    2  1.0
## [3,]    0    0 -0.5
```

1.5.3. Solucion vía LU

```
forward_sub <- function(L, b) {
  n <- nrow(L); y <- numeric(n)
  for (i in 1:n) {y[i] <- (b[i] - sum(L[i, 1:(i - 1)] * y[1:(i - 1)]))}
  y
}
```

```
# x en Ux = y
back_sub <- function(U, y, tol = 1e-12) {
  n <- nrow(U); x <- numeric(n)
  for (i in n:1) {
    s <- if (i == n) 0 else sum(U[i, (i + 1):n] * x[(i + 1):n])
    if (abs(U[i, i]) < tol) stop(sprintf("Pivote ~0 en U[%d,%d].", i, i))
    x[i] <- (y - s) / U[i, i]
  }
  x
}
```

1.5.3.1. Resolucion sin pivoteo

```
solve_lu_simple <- function(A, b, tol = 1e-12) {
  lu <- lu_simple(A, tol = tol);
  y <- forward_sub(lu$L, b)
  x <- back_sub(lu$U, y, tol = tol)
  x
}
```

1.5.3.2. Resolucion con pivoteo

```
solve_lu_piv_parcial <- function(A, b, tol = 1e-12) {
  lu <- lu_piv_parcial(A, tol = tol);
  Pb <- lu$P %*% b
  y <- forward_sub(lu$L, as.vector(Pb))
  x <- back_sub(lu$U, y, tol = tol)
  x
}
```

1.5.3.3. Ejemplo

```
## Warning in x[i] <- (y - s)/U[i, i]: número de elementos para sustituir no es un
## múltiplo de la longitud del reemplazo
```

```
## Warning in x[i] <- (y - s)/U[i, i]: número de elementos para sustituir no es un
## múltiplo de la longitud del reemplazo
## Warning in x[i] <- (y - s)/U[i, i]: número de elementos para sustituir no es un
## múltiplo de la longitud del reemplazo
## [1] 6.0 -4.5 6.0
```

1.6. Factorización de Cholesky

```
tolerancia <- 1e-12
cholesky_fact <- function(A,tolerancia) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A)
  if (ncol(A) != n) stop("A debe ser cuadrada.")
  if (!all(abs(A - t(A)) < tolerancia)) stop("A debe ser simétrica.")
  L <- matrix(0, n, n)
  for (j in 1:n) {
    suma <- sum(L[j, 1:(j-1)]^2)
    val <- A[j, j] - suma
    if (val <= 0) stop("A no es definida positiva (falló Cholesky).")
    L[j, j] <- sqrt(val)
    if (j < n) {
      for (i in (j+1):n) {
        suma <- sum(L[i, 1:(j-1)] * L[j, 1:(j-1)])
        L[i, j] <- (A[i, j] - suma) / L[j, j]
      }
    }
    cat(sprintf("Paso j=%d\n", j))
    print(L)
  }
  return(L)
}
```

1.6.1. Ejemplo

```
## Paso j=1
##      [,1] [,2] [,3]
## [1,] 2    0    0
## [2,] 6    0    0
## [3,] -8   0    0
## Paso j=2
##      [,1] [,2] [,3]
## [1,] 2    0    0
## [2,] 6    1    0
## [3,] -8   5    0
## Paso j=3
##      [,1] [,2] [,3]
## [1,] 2    0    0
## [2,] 6    1    0
## [3,] -8   5    3
##      [,1] [,2] [,3]
```

```
## [1,] 2 0 0
## [2,] 6 1 0
## [3,] -8 5 3
```

1.7. Solución vía Cholesky

```
tolerancia <- 1e-12
solve_cholesky <- function(A, b, tolerancia) {
  L <- cholesky_fact(A, tolerancia)
  y <- forward_sub(L, b)
  x <- back_sub(t(L), y, tolerancia)
  x
}
```

1.7.1. Ejemplo

```
## Paso j=1
##      [,1] [,2] [,3]
## [1,] 2 0 0
## [2,] 6 0 0
## [3,] -8 0 0
## Paso j=2
##      [,1] [,2] [,3]
## [1,] 2 0 0
## [2,] 6 1 0
## [3,] -8 5 0
## Paso j=3
##      [,1] [,2] [,3]
## [1,] 2 0 0
## [2,] 6 1 0
## [3,] -8 5 3

## Warning in x[i] <- (y - s)/U[i, i]: número de elementos para sustituir no es un
## múltiplo de la longitud del reemplazo
## Warning in x[i] <- (y - s)/U[i, i]: número de elementos para sustituir no es un
## múltiplo de la longitud del reemplazo
## Warning in x[i] <- (y - s)/U[i, i]: número de elementos para sustituir no es un
## múltiplo de la longitud del reemplazo

## [1] 3.8333333 -0.6666667 0.3333333

##      [,1]
## [1,] 2.000000e+00
## [2,] 7.000000e+00
## [3,] -7.105427e-15
```

Capítulo 2

Métodos Iterativos

2.1. Gauss Jacobi

```
jacobi <- function(A, b, x0 = NULL, tol = 1e-8, maxiter = 1000) {  
  if (!is.matrix(A)) stop("A debe ser una matriz.")  
  n <- nrow(A)  
  if (ncol(A) != n) stop("A debe ser cuadrada.")  
  if (length(b) != n) stop("Dimensiones de b incompatibles.")  
  if (is.null(x0)) x0 <- rep(0, n)  
  x <- x0; D <- diag(diag(A)); R <- A - D;  
  for (k in 1:maxiter) {  
    x_new <- (b - R %*% x) / diag(D)  
    cat(sprintf("Iter%d: %s\n", k, paste(round(x_new, 6), collapse = " ")))  
    if (sqrt(sum((x_new - x)^2)) < tol) {return(list(x = as.vector(x_new), iter = k, convergencia = TRUE))}  
    x <- x_new  
  }  
  list(x = as.vector(x), iter = maxiter, convergencia = FALSE)  
}
```

2.1.1. Ejemplo

```
## Iter 1: 3.75 2.5 3.333333  
## Iter 2: 4.375 4.270833 4.166667  
## Iter 3: 4.817708 4.635417 4.756944  
## Iter 4: 4.908854 4.893663 4.878472  
## Iter 5: 4.973416 4.946832 4.964554  
## Iter 6: 4.986708 4.984493 4.982277  
## Iter 7: 4.996123 4.992246 4.994831  
## Iter 8: 4.998062 4.997738 4.997415  
## Iter 9: 4.999435 4.998869 4.999246  
## Iter 10: 4.999717 4.99967 4.999623  
## Iter 11: 4.999918 4.999835 4.99989  
## Iter 12: 4.999959 4.999952 4.999945  
## Iter 13: 4.999988 4.999976 4.999984  
## Iter 14: 4.999994 4.999993 4.999992  
## Iter 15: 4.999998 4.999996 4.999998
```

```
## Iter 16: 4.999999 4.999999 4.999999
## Iter 17: 5 4.999999 5
## Iter 18: 5 5 5
## Iter 19: 5 5 5
## Iter 20: 5 5 5
## Iter 21: 5 5 5
## Iter 22: 5 5 5

## [1] 5 5 5
```

2.2. Gauss Seidel

```
gauss_seidel <- function(A, b, x0 = NULL, tol = 1e-8, maxiter = 1000, verbose = FALSE) {
  if (!is.matrix(A)) stop("A debe ser una matriz.")
  n <- nrow(A)
  if (ncol(A) != n) stop("A debe ser cuadrada.")
  if (length(b) != n) stop("Dimensiones de b incompatibles.")
  x <- if (is.null(x0)) rep(0, n) else as.numeric(x0)
  if (length(x) != n) stop("Dimensión de x0 incompatible con A.")
  for (k in 1:maxiter) {
    x_old <- x
    for (i in 1:n) {
      aii <- A[i, i]
      if (abs(aii) < .Machine$double.eps) {stop(sprintf("A[%d,%d] = 0: Gauss-Seidel No se puede aplicar",
s1 <- if (i > 1) sum(A[i, 1:(i - 1)] * x[1:(i - 1)]) else 0
s2 <- if (i < n) sum(A[i, (i + 1):n] * x_old[(i + 1):n]) else 0
      x[i] <- (b[i] - s1 - s2) / aii
    }
    dx <- sqrt(sum((x - x_old)^2)); res <- sqrt(sum((b - A %*% x)^2))
    cat(sprintf("Iter%4d |dx|=%.3e |res|=%.3e x=%s\n", k, dx, res, paste(round(x, 6), collapse=" ")))
    if (dx < tol && res < tol) {
      return(list(x = as.vector(x), iter = k, convergencia = TRUE, delta = dx, residuo = res))
    }
  }
  list(x = as.vector(x), iter = maxiter, convergencia = FALSE,
    delta = sqrt(sum((x - x_old)^2)), residuo = sqrt(sum((b - A %*% x)^2)))
}
```

2.2.1. Ejemplo

```
## Iter    1 |dx|=6.778e+00 |res|=5.646e+00 x=3.75 3.4375 4.479167
## Iter    2 |dx|=1.649e+00 |res|=1.407e+00 x=4.609375 4.772135 4.924045
## Iter    3 |dx|=3.917e-01 |res|=2.052e-01 x=4.943034 4.96677 4.988923
## Iter    4 |dx|=5.712e-02 |res|=2.992e-02 x=4.991692 4.995154 4.998385
## Iter    5 |dx|=8.330e-03 |res|=4.363e-03 x=4.998788 4.999293 4.999764
## Iter    6 |dx|=1.215e-03 |res|=6.363e-04 x=4.999823 4.999897 4.999966
## Iter    7 |dx|=1.772e-04 |res|=9.280e-05 x=4.999974 4.999985 4.999995
## Iter    8 |dx|=2.584e-05 |res|=1.353e-05 x=4.999996 4.999998 4.999999
## Iter    9 |dx|=3.768e-06 |res|=1.974e-06 x=4.999999 5 5
## Iter   10 |dx|=5.495e-07 |res|=2.878e-07 x=5 5 5
## Iter   11 |dx|=8.013e-08 |res|=4.197e-08 x=5 5 5
```



```
## Iter    12 |dx|=1.169e-08 |res|=6.121e-09 x=5 5 5
## Iter    13 |dx|=1.704e-09 |res|=8.926e-10 x=5 5 5
## [1] 5 5 5
```

